

Project 3: 地图导航

叶锦灏 软件工程 24300750085

摘要

本实验实现了一个基于图结构的路径规划系统，用于在给定地图中查询任意起点与终点之间的最优路径。地图由若干地点（节点）及其之间的连接关系（边）构成，每条边带有一个非负权重，用以表示距离或行程代价。系统采用 **n**次 **Dijkstra** 算法进行预处理，计算并存储任意两点之间的最短距离信息，从而支持高效的多次查询。在查询阶段，综合考虑不同的路径优化目标，提供最短距离路径（Shortest Distance Path）、最少跳数路径（Least-hop Path）、其他备选路径（Alternative Paths）共最多三条路径以供用户参考，并对每条路径给出路径链、总距离、跳数以及相对绕行程度等指标，以便对不同路径方案进行对比分析。

算法选择与实现

图的数据结构选择

在本实验中，地图被建模为一个无向加权图。综合考虑图规模较小、查询频繁以及算法实现的清晰性，采用**邻接表（adjacency list）**作为图的基本存储结构。

具体而言，每个节点对应一个结构体，所有节点用一个 **vector** 存储，节点id直接对应 **vector** 的下标；节点内部维护一个邻接边列表，边中记录目标节点编号及对应的权重。核心代码如下：

```

struct adjEdge{
    int to;
    double w;
};

struct node{
    std::string name;
    std::vector<adjEdge> adj;
};

class graph{
    // 无关部分省略
private:
    std::unordered_map<std::string, int> name2id; // 用来存储地点名称到id的映射，其底层
    std::vector<std::string> id2name;
    std::vector<node> V;

}

```

之所以选择邻接表，是因为在图相对稀疏的情况下比邻接矩阵有更好的空间效率，可以避免不必要的存储开销，同时也便于在 Dijkstra、BFS 等算法中高效遍历相邻节点。在本实验中，共有26个节点，39条边，有网络密度 $D = \frac{2|E|}{|V|(|V| - 1)} = 0.12$ ，属于较稀疏图，适用于邻接链表。

此外，为了支持以地点名称进行查询，系统使用哈希表（ `unordered_map` ）维护地点名称到节点编号的映射关系，并通过 `vector` 反向维护编号到名称的映射。

抽象出的地图类 `class graph` 另外提供了这些公共接口：

```

    void addEdge(std::string src, std::string dest, double weight)
// 用于初始化地图
    int vsize()
// 获取节点数
    const std::vector<adjEdge>& neighbors(int u)
// 获取节点邻居
    const std::string& name(int id)
// 用名字查id
    int idOf(const std::string& name)
// 用id查名字

```

All Pairs Shortest Path 预计算：n次 Dijkstra 算法

为了支持多次路径查询，系统在加载地图后对所有节点对进行最短路径预计算，采用的方法是：对每个节点执行一次 **Dijkstra** 算法，即 n 次 Dijkstra。

在算法选择上，没有采用 Floyd–Warshall 或 Johnson 算法，主要原因如下：

- Floyd–Warshall 算法时间复杂度为 ($O(n^3)$)，虽然在节点数较小时可行，但扩展性较差；
- Johnson 算法主要用于含负权边但无负环的场景，而本实验中的边权均为非负实数，因此没有必要引入额外的 Bellman–Ford 预处理步骤；
- n 次 Dijkstra 在稀疏图中具有更优的时间复杂度 ($O(n \cdot m \log n)$)，同时实现相对直接，适合本实验的图规模和数据特点。

在实现过程中，使用优先队列（ `priority_queue` ）作为最小堆。由于标准库未提供 `decrease_key` 操作，算法采用“重复插入 + 过期状态过滤”的方式进行替代：当发现更短路径时，将新的状态插入队列；在弹出队列元素时，若其距离大于当前记录的最短距离，则直接忽略。这里我一开始认为，可以通过维护一个颜色参数来判断节点是否已经被处理完毕来代替距离判断的条件。但这实际上是不可行的，因为在松弛过程中多次插入，实际上可能导致节点被提前染色。一个体会是，算法的理论和编程语言的实现之间还是有着比较大的鸿沟。Dijkstra算法核心代码如下：

```

void dijkstra(const graph& G, int s, vector<double>& dist, vector<int>& parent){
    const int WHITE = 0;
    const int BLACK = 1;
    const int n = G.vsize();
    vector<int> color(n, WHITE);
    // vector<int> parent(n, -1);
    // vector<double> dist(n, INF);
    // 这里写错了，既然目的是让外面的容器收到节点，那就不应该在内层重新声明

    dist.assign(n, INF);
    parent.assign(n, -1);

    dist[s] = 0.0;

    using P = pair<double, int>;
    priority_queue<P, vector<P>, greater<P>> Q;
    Q.push({dist[s], s});

    while(!Q.empty()){
        auto [d,v] = Q.top(); Q.pop();
        if (d > dist[v]) continue; // 这一步是为了适应decrease_key的等价实现
        color[v] = BLACK;

        for(auto& e : G.neighbors(v)){
            int u = e.to;
            double w = e.w;
            if(color[u] != BLACK){ // 这个条件判断实际上是不需要的，因为即使不保留它，BI
                // 而且跳过这些节点对性能的优化可以忽略不计
                if(dist[u] > dist[v] + w){
                    dist[u] = dist[v] + w;
                    parent[u] = v;
                    Q.push({dist[u], u});
                    // 这里其实应该是decrease_key操作，但是标准库的priority_queue没有提
                    // GPT老师说，这里的替代策略是，用重复的Q.push(d, u) 这实际上意味着
                    // 会存在多个重复的条目，有新的有旧的，新的d一定比旧的小，而dist[]
                    // 所以一种实现策略是，在每次top+pop的时候，检查d和dist的大小关系。
                    // 但是我的策略是，直接标记节点的颜色，取出去之后标记为黑色，这样后续
                }
            }
        }
    }
}

```

基于此实现的n次Dijkstra算法如下：

```
void all_dijkstra(const graph& G, vector<vector<double>>& dist, vector<vector<int>>& parent){
    const int n = G.vsize();
    dist.resize(n);
    parent.resize(n);
    for(int i = 0; i < n; i++){
        dijkstra(G, i, dist[i], parent[i]);
    }
}
```

预计算阶段不仅存储最短距离矩阵，还额外维护前驱节点矩阵，用于后续路径的恢复。相应地，编写了在前驱矩阵中递归查询打印路径的函数。这一设计以 ($O(n^2)$) 的空间复杂度换取了查询阶段的高效性。

最少跳数路径：广度优先搜索（BFS）

除最短距离路径外，系统额外计算最少跳数路径，即在忽略边权的情况下，从起点到终点经过边数最少的路径。该路径通过**广度优先搜索（Breadth-First Search, BFS）**获得。

BFS 在无权图中能够保证找到最少边数的路径，因此在本实验中将每条边视为单位权重即可直接应用。最少跳数路径不一定具有最短距离，但在实际场景中，它往往对应路径结构更简单、经过节点更少的方案。

通过同时输出最短距离路径和最少跳数路径，系统能够展示不同优化目标之间的权衡关系，为路径选择提供更全面的参考。

备选路径：Yen's K-shortest Loopless Paths

为了进一步提供结构上不同的备选路径，系统引入 Yen's K-shortest loopless paths 算法。该算法用于在给定起点和终点之间，寻找多条不含环的最短简单路径。

算法的基本思想是：以最短路径为基准，依次选择路径中的某个节点作为偏离点（spur node），通过临时禁用部分边或节点，强制路径在该位置发生偏离，从而生成新的候选路径。在所有候选路径中，选择代价最小者作为下一条备选路径。

在本实验中，Yen 算法并不用于穷举大量路径，而是作为生成少量高质量替代路径的工具。结合最短距离路径和最少跳数路径，系统最终最多输出三条路径，并通过标签区分其优化目标（最短距离、最少跳数或备选路径），在保证结果多样性的同时避免输出过度绕行的路径。由于该算法的部分不是实验的重点，所以直接让ChatGPT老师写了这部分代码。

路径的评价指标

为了让用户评价不同路径的优劣，程序还返回下面三个路径的评价指标：

- **(1) 总路径长度 (Total Distance)**
路径中所有边权之和。
- **(2) 跳数 (Hop Count)**
路径中经过的边数（节点数 - 1），反映路径的结构复杂度。跳数较少的路径通常结构更简单，对应更少的拐弯、红绿灯等。
- **(3) 相对绕行程度 (Detour Ratio)**
衡量路径相对于最短路径的冗余程度，定义为：

$$\text{Detour Ratio} = \frac{\text{Path Length}}{\text{Shortest Path Length}}$$

该指标用于比较不同路径的相对效率，数值越接近 1 表示路径越接近最优。

运行示例

程序运行示例：

```
Map loaded. Nodes = 26
APSP preprocessing finished.
Enter queries (start end), Ctrl+D to exit:
```

A P

```
3 routes A -> P (best=17.63)
#1 17.63 detour=1.00 hops=6 A->F->G->J->N->O->P (shortest time)
#2 21.14 detour=1.20 hops=4 A->B->E->L->P (least hop)
#3 18.83 detour=1.07 hops=6 A->F->G->H->K->O->P (alternative)
```

B F

```
3 routes B -> F (best=11.21)
#1 11.21 detour=1.00 hops=5 B->E->D->H->G->F (shortest time)
#2 12.43 detour=1.11 hops=2 B->A->F (least hop)
#3 11.87 detour=1.06 hops=5 B->E->D->C->G->F (alternative)
```

C T

```
3 routes C -> T (best=13.55)
#1 13.55 detour=1.00 hops=5 C->G->J->N->O->T (shortest time)
#2 14.79 detour=1.09 hops=5 C->D->H->K->O->T (least hop)
#3 14.70 detour=1.08 hops=5 C->G->J->N->S->T (alternative)
```

Z A

```
3 routes Z -> A (best=23.66)
#1 23.66 detour=1.00 hops=9 Z->Y->U->T->O->N->J->G->F->A (shortest time)
#2 27.80 detour=1.17 hops=7 Z->Y->U->P->L->E->B->A (least hop)
#3 24.29 detour=1.03 hops=9 Z->Y->U->P->O->N->J->G->F->A (alternative)
```

M O

```
3 routes M -> O (best=7.41)
#1 7.41 detour=1.00 hops=3 M->Q->P->O (shortest time)
#2 8.07 detour=1.09 hops=3 M->L->K->O (least hop)
#3 7.75 detour=1.05 hops=3 M->L->P->O (alternative)
```

总结

本实验围绕加权图上的路径规划问题，设计并实现了一个支持多源查询的路径搜索系统。通过采用 n 次 Dijkstra 算法进行全局预处理，系统在保证计算正确性的同时，实现了高效的在线查询。

在路径结果的呈现上，实验不仅给出了最短距离路径，还进一步引入最少跳数路径以及结构不同的备选路径，并通过总路径长度、跳数和相对绕行程度等指标对不同路径方案进行评价。这种多指标的分析方式表明，即使在仅有路径长度信息的条件下，仍可以从不同角度对路径质量进行比较和解释。

实验结果显示，不同优化目标之间往往存在权衡关系，部分节点对的最短路径对特定边具有高度依赖性，从而导致合理的替代路径数量有限。总体而言，本实验在经典最短路径算法的基础上，结合工程实现与分析视角，对路径规划问题进行了较为全面的探索，并为后续引入更多路径属性或多目标优化提供了良好的扩展基础。